

9

内部排序

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang

内容要点

- 概述
- 基本排序
- 归并排序
- 快速排序
- 堆排序
- 计数排序
- 树形选择排序
- 基数排序

目录

1	概述
2	基本排序
3	分治法排序
4	树形选择排序
5	非比较排序

排序的定义

- 对于数据表 $\mathcal{R} = \{r_1, r_2, \dots, r_n\}$ ，试确定 $1, 2, \dots, n$ 的一种排列 $p_1, p_2, \dots, p_n$ ，使数据表 $\mathcal{R}$ 满足非递减关系：

$$r_{p_1} \leq r_{p_2} \leq \dots \leq r_{p_n}$$

- 即数据表 $\{r_{p_1}, r_{p_2}, \dots, r_{p_n}\}$ 递增有序。
- 对于数据表 $\mathcal{R}$ ，如果有 $r_i = r_j$ ($i < j$)，如果排序前 r_i 领先于 r_j ，排序后 r_i 仍然领先于 r_j ，则称该排序方法是稳定的。否则称该排序方法是不稳定的。

设 $\mathcal{R}$ 排序前 (56, 34, 47, 23, 66, 18, 47)。

- ✓ 如果排序后得到 (18, 23, 34, 47, 47, 56, 66)，则称该排序方法是稳定的；
- ✓ 如果排序后得到 (18, 23, 34, 47, 47, 56, 66)，则称该排序方法是不稳定的。

排序的分类

• 按策略流派分类

策略流派	包含的排序算法	核心思想（一句话概括）
插入/交换/选择 （朴素基础）	基本排序（指冒泡、选择、插入）	利用相邻元素交换或不断选择极值，逐步缩小无序区间，时间复杂度 $O(n^2)$ 。
分治法	归并排序、快速排序	将大数组递归拆分为独立子问题，分别求解后再合并（归并），或先分区再递归（快排）。
树形选择 （选择排序的进化）	堆排序、树形选择排序（锦标赛排序）	利用完全二叉树（堆）或锦标赛树的结构，快速找出极值，避免重复比较。
线性/分布排序 （非比较类）	计数排序、基数排序	不通过比较关键字大小，而是通过分类、计数或按位分配直接确定元素最终位置。

排序的分类

• 按是否基于比较分类（决定时间复杂度下界）

类别	包含算法	时间复杂度下界	核心特征
基于比较的排序	基本排序、归并排序、快速排序、堆排序、树形选择排序	存在下界 $\Omega(n \log n)$	只依赖元素间的大小比较（大于、小于、等于）来决策顺序。
非基于比较的排序	计数排序、基数排序	可突破 $O(n \log n)$ ，达到 $O(n)$	依赖数据本身的数值范围（如整数范围 k ）或位数（如 d 位数字）。

排序的分类

• 按“时间复杂度特性”分类（工程选型速查）

时间复杂度	包含算法	备注
$O(n^2)$	基本排序	数据量小（ < 1000 ）时可用，简单且稳定性可控。
$O(n\log n)$	归并排序、快速排序（平均）、堆排序、树形选择排序	工业界主力。快排最快（常数因子小），归并稳定，堆排序最省内存空间（原地）。
$O(n)$ （线性）	计数排序、基数排序	有苛刻前提：数据必须是整数且范围有限（计数），或位数已知（基数）。

目录

1	概述
2	基本排序
3	分治法排序
4	树形选择排序
5	非比较排序

比较式排序

- 两种基本操作：
 - 比较两个关键字的大小
 - 交换两个记录的位置
- 两类数据区：

有序区

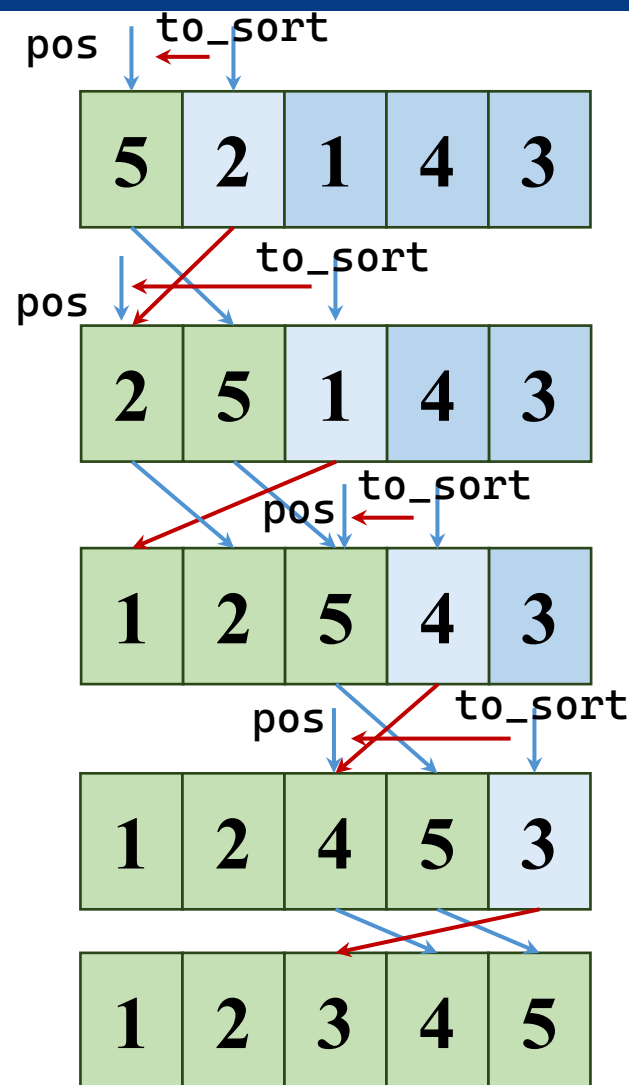
无序区

插入排序

- 算法

- 依次处理无序区首元素
- 从后往前找到位置
 - 越界或首次小于关键字的位置
- 有序区每个元素后移
- 关键字复制到指定位置

- 依次从无序区取首元素，在有序区寻找插入位置的方法，称插入排序



插入排序

• 算法

– 依次处理无序区首元素

– 从后往前找到位置

▪ 越界或首次小于关键字的位置

– 有序区每个元素后移

– 关键字复制到指定位置

```
for (i = 1; i < count; ++i) {  
    int value = array[i], p = i - 1;  
    while (p >= 0 && array[p] > value) {  
        array[p + 1] = array[p];  
        --p;  
    }  
    array[p + 1] = value;  
}
```

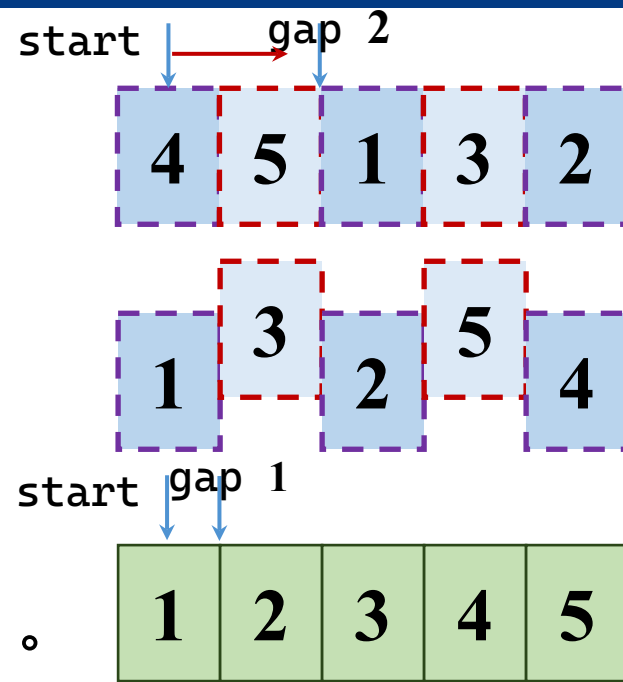
```
void insertionSort(int array[], int count) {  
    int index;  
  
    for (index = 1; index < count; ++index) {  
        int value = array[index];  
        int position = index - 1;  
  
        while (position >= 0 && array[position] > value) {  
            array[position + 1] = array[position];  
            --position;  
        }  
        array[position + 1] = value;  
    }  
}
```

插入排序的时间分析

- 最好情况(关键字在数据表中有序)：
 - 比较次数： $n-1$
 - 移动次数： 0
- 最坏情况(关键字在数据表中逆序有序)：
 - 比较次数： $\sum_{i=2}^n i = \frac{(n+2)(n-1)}{2}$
 - 移动次数： $\sum_{i=2}^n (i+1) = \frac{(n+4)(n-1)}{2}$

希尔排序

- 希尔 (Shell) 排序
 - 又称缩小增量排序
- 基本思想：
 - 将整个数据表分成m个子序列，对每个子序列分别进行插入排序。
 - 逐步减少子序列数m，直至m=1为止。
- 时间复杂度： $O(n^2/m)$



希尔排序

```
for (gap = count / 2; gap > 0; gap /= 2) {  
    for (int start = 0; start < gap; ++start) {
```

start + gap

i += gap

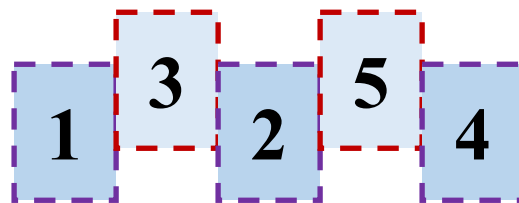
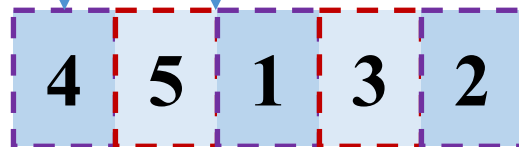
p = i - gap;

```
for (i = 1; i < count; ++i) {  
    int value = array[i];  
    int p = index - 1;  
    while (p >= 0 && array[p] > value) {  
        array[p + 1] = array[p];  
        --p;  
    }  
    array[p + 1] = value;  
}
```

array[p + gap] = value;

p -= gap;

start → gap 2



start → gap 1



array[p + gap] = array[p];

选择排序

- 算法：直接选数不插入

- 外层循环控制无序区的范围

```
for (begin = 0; begin < count - 1; ++begin) {
```

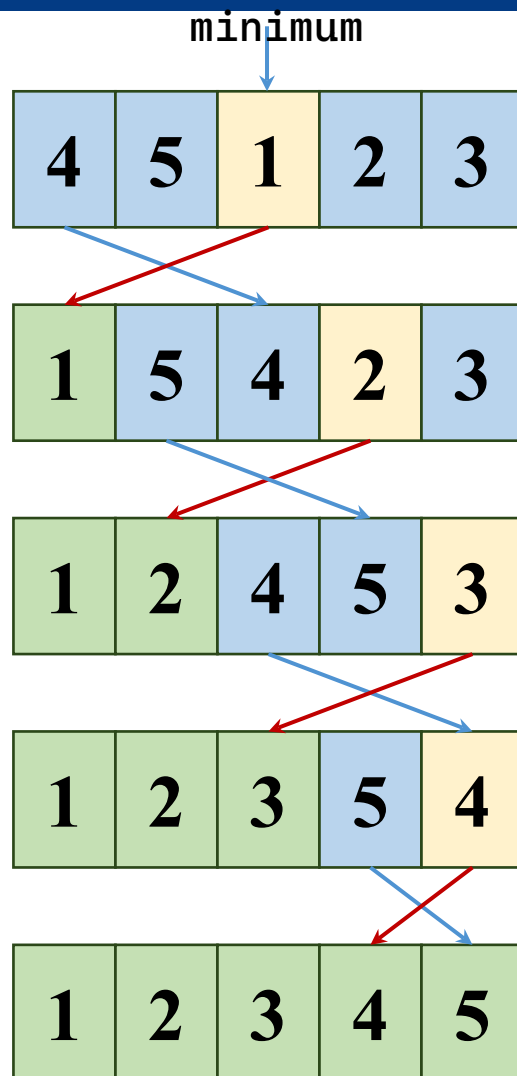
- 内层循环取得将最小值

```
    int minimum = begin, i;  
    for (i = begin + 1; i < count; ++i) {  
        if (array[i] < array[minimum])  
            minimum = i;
```

- 与无序区首位置换

```
    if (minimum != begin)  
        swap(&array[begin], &array[minimum]);
```

```
}
```



```

static void swap(int* left, int* right) {
    int temporary = *left;
    *left = *right;
    *right = temporary;
}

void selectionSort(int array[], int count) {
    int begin;
    for (begin = 0; begin < count - 1; ++begin) {
        int minimum = begin;
        int index;
        for (index = begin + 1; index < count; ++index) {
            if (array[index] < array[minimum]) {
                minimum = index;
            }
        }
        if (minimum != begin) {
            swap(&array[begin], &array[minimum]);
        }
    }
}

```

冒泡排序

- 算法：把轻的泡泡浮上来

- 外层循环控制无序区的范围

```
for (unsorted = count; unsorted > 1; --unsorted) {
```

- 内层循环比较轻重

```
    for (i = 1; i < unsorted; ++i) {
```

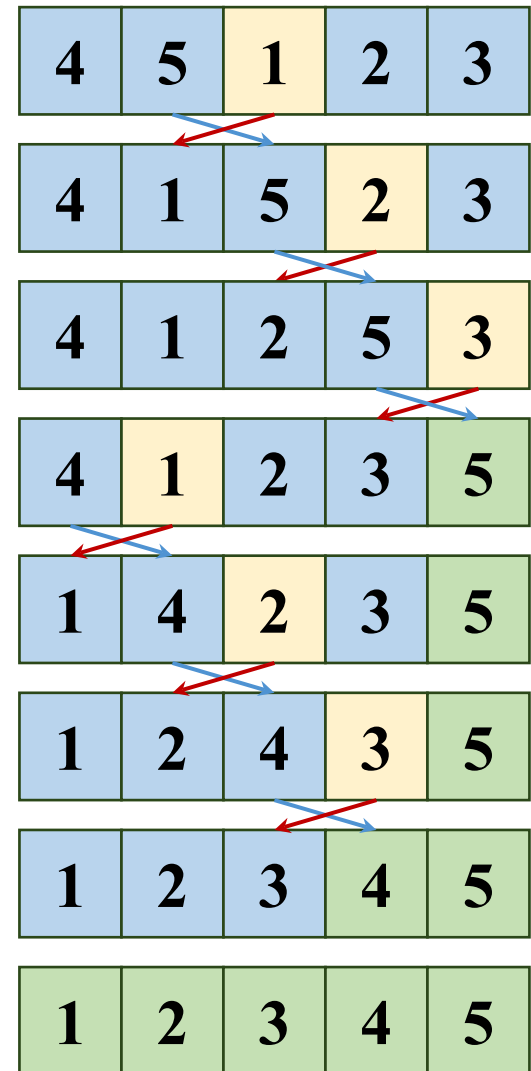
- 不对就交换

```
        if (array[i - 1] > array[i]) {  
            swap(&array[i - 1], &array[i]);
```

```
        }  
    }
```

- 加速

- 如果某一层没有交换，则算法已完成



```

void bubbleSort(int array[], int count) {
    int unsorted;

    for (unsorted = count; unsorted > 1; --unsorted) {
        int index;
        bool changed = false;

        for (index = 1; index < unsorted; ++index) {
            if (array[index - 1] > array[index]) {
                swap(&array[index - 1], &array[index]);
                changed = true;
            }
        }
        if (!changed) {
            return;
        }
    }
}

```

冒泡排序：性能分析

- 最好情况(关键字在数据表中递增有序)：

- 只需进行一趟冒泡。

- 比较的次数： $n-1$ 移动的次数：0

- 最坏情况(关键字在数据表中递减有序)：

- 需进行 $n-1$ 趟冒泡。

- 比较的次数： 移动的次数：

$$\sum_{i=n}^2 (i-1) = \frac{n(n-1)}{2}$$

$$3 \cdot \sum_{i=n}^2 (i-1) = \frac{3n(n-1)}{2}$$

```

void bidirectionalBubbleSort(int array[], int count) {
    int left = 0, right = count - 1;
    while (left < right) {
        int lastForward = left, lastBackward = right;
        for (int j = left; j < right; ++j) {
            /* 从左向右扫描, 将较大的数往后冒泡 */
            if (array[j] > array[j + 1]) {
                swap(&array[j], &array[j + 1]);
                lastForward = j;
            }
            /* 从右向左扫描, 将较小的数往前冒泡 */
            int k = right + left - j; /* 对称位置的索引 */
            if (array[k] < array[k - 1]) {
                swap(&array[k], &array[k - 1]);
                lastBackward = k;
            }
        }
        /* 收缩无序区间 */
        right = lastForward; left = lastBackward;
    }
}

```

地精排序

- 算法

- 遍历数组

```
while (pos < count) {
```

- 如果有序区未发现乱序

- 只有一个元素

```
if (pos == 0{
```

- 顺序是对的

```
|| array[pos] >= array[pos - 1])
```

- 则前进

```
++pos;
```

- 否则交换

```
else {  
    swap(&array[pos], &array[pos - 1]);
```

- 有序区后撤

```
--pos;
```

```
}
```

```
}
```

```

void gnomeSort(int array[], int count) {
    int pos = 0;

    while (pos < count) {
        if (pos == 0 || array[pos] >= array[pos - 1]) {
            ++pos;
        } else {
            swap(&array[pos], &array[pos - 1]);
            --pos;
        }
    }
}

```

本节小结

- 基本排序：

- 插入排序， $O(n^2)$
- 希尔排序，时间复杂度优于插入排序
- 选择排序， $O(n^2)$
- 冒泡排序， $O(n^2)$ ，实际效率可能较高
- 地精排序， $O(n^2)$ ，单重循环的排序算法

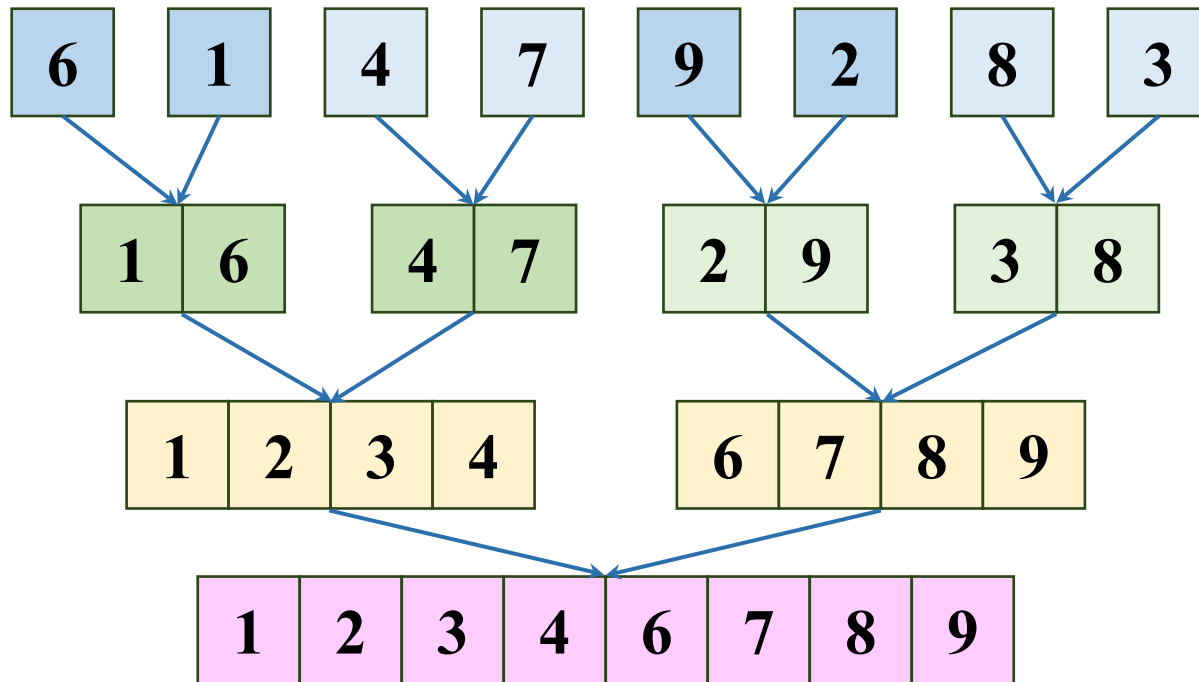
目录

	3	分治法排序
	3.1	归并排序
	3.2	快速排序
	4	树形选择排序
	5	非比较排序

归并排序

- 归并排序

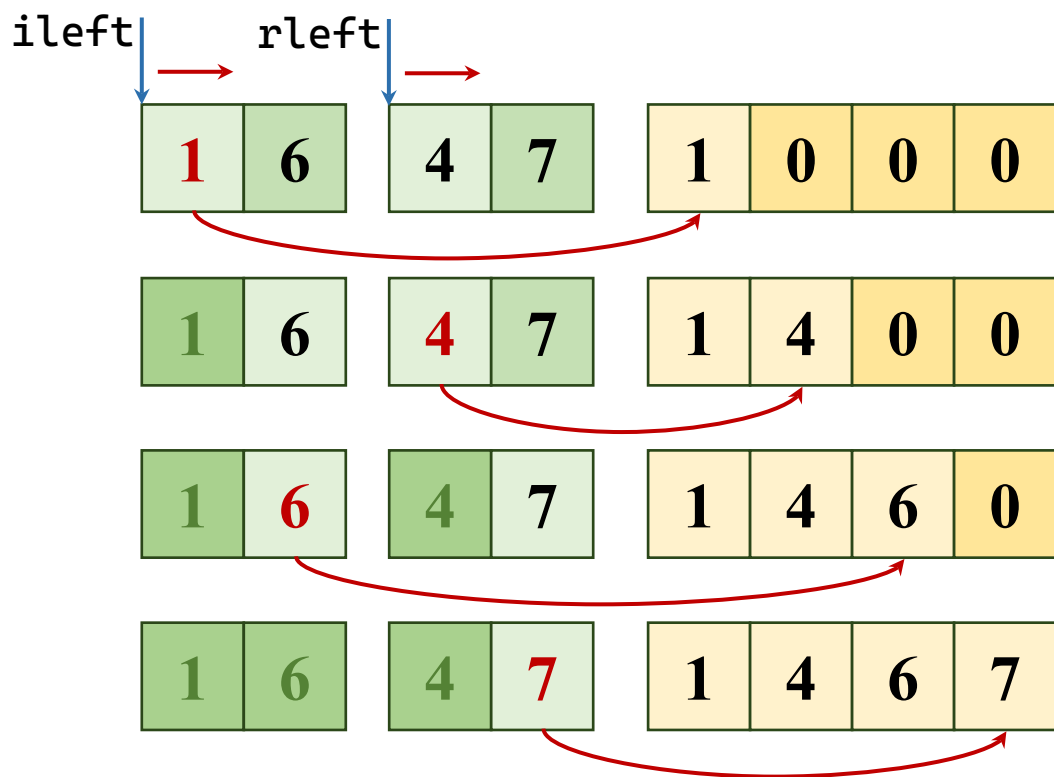
- 通过将两个(2路)或两个以上的有序(子)序列合并为一个有序序列的方法，对序列进行排序。



归并排序

• 合并算法

- 左右半边从左到右遍历
- 当前位置左边小，存左
- 当前位置右边小，存右
- 左边完了全存右
- 右边完了全存左



- 为了简化，前面存在临时数组，后续复制回去

归并排序

• 合并算法

– 左右半边从左到右遍历

– 当前位置左边小，存左

– 当前位置右边小，存右

– 左边完了全存右

– 右边完了全存左

– 为了简化，前面存在临时数组，后续复制回去

```
int ileft = left, iright = mid + 1, target = left;
```

```
while (ileft <= mid && iright <= right)
```

```
if (array[ileft] <= array[iright])  
    aux[target++] = array[ileft++];  
else  
    aux[target++] = array[iright++];
```

```
while (ileft <= middle)  
    auxiliary[target++] = array[ileft++];
```

```
while (iright <= middle)  
    auxiliary[target++] = array[iright++];
```

```
for (target = left; target <= right; ++target)  
    array[target] = auxiliary[target];
```

归并排序

- 总体算法

- 递归，每一层都有
- 先排序左半部，再排序右半部，最后合并两半部分

```
void mergeSortRange(int array[], int aux[], int left, int right) {  
    int middle;  
    if (left >= right)  
        return;  
    middle = left + (right - left) / 2;  
    mergeSortRange(array, auxiliary, left, middle);  
    mergeSortRange(array, auxiliary, middle + 1, right);  
    merge(array, auxiliary, left, middle, right);  
}
```

- 排序接口

```
mergeSortRange(array, auxiliary, 0, count - 1);
```

```

void mergeSort(int array[], int count) {
    int* auxiliary;

    if (count < 2) {
        return;
    }
    auxiliary = (int*)malloc((size_t)count * sizeof(int));
    if (auxiliary == NULL) {
        perror("无法分配归并排序辅助数组");
        exit(EXIT_FAILURE);
    }
    mergeSortRange(array, auxiliary, 0, count - 1);
    free(auxiliary);
}

```

目录

	3	分治法排序
	3.1	归并排序
	3.2	快速排序
	4	树形选择排序
	5	非比较排序

快速排序

- 基本思想

- 通过一趟快速排序将待排数据分割成独立的左中右三个部分，其中，中部分只含一个数，左边部分的所有关键字都比这个数小，右边部分的所有关键字都比这个数大。
- 然后再按此方法对这两部分记录分别进行快速排序，直到所有记录整理成有序序列。

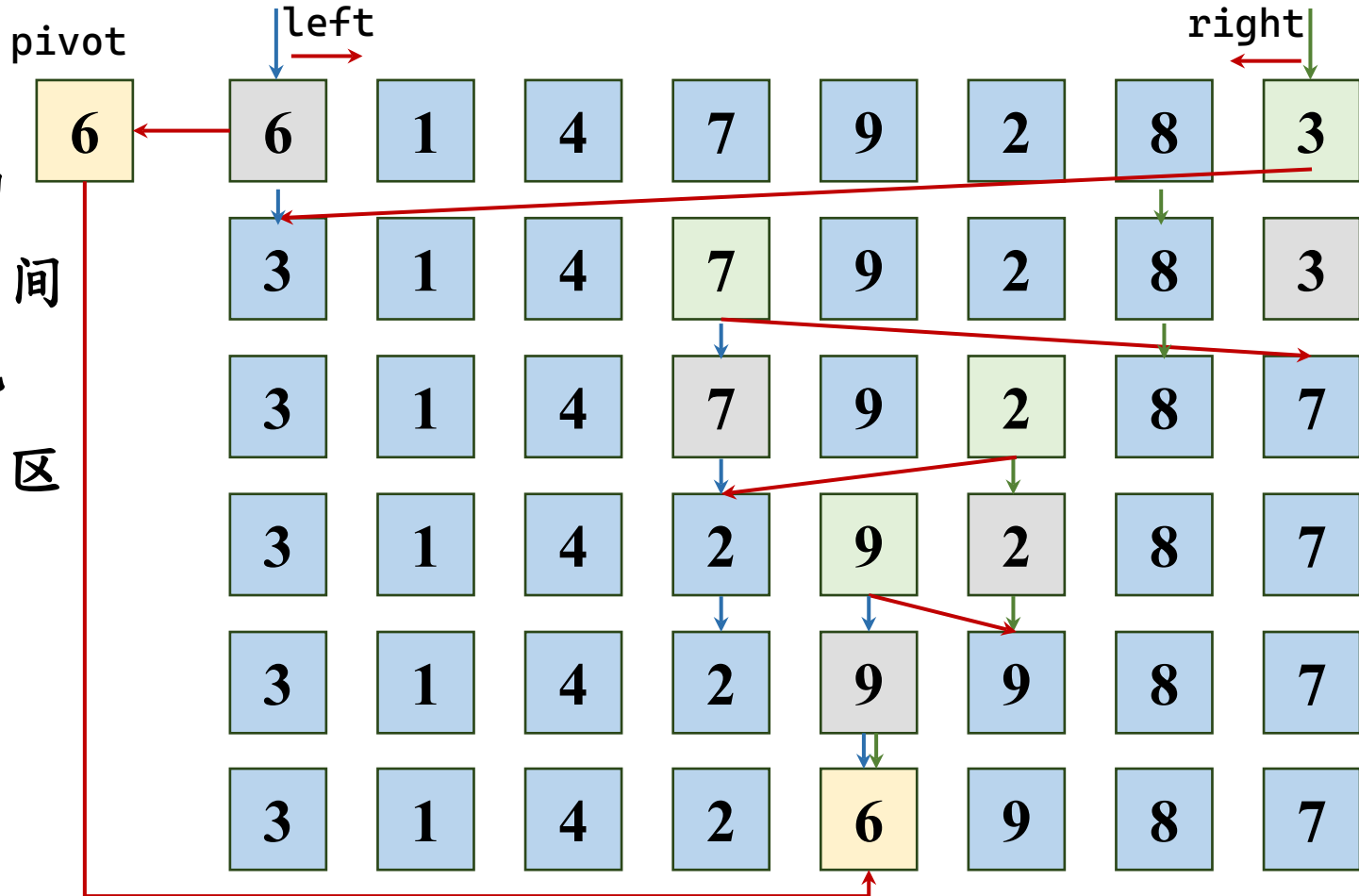
- 枢轴的选取

- 选取末尾元素实现更简洁，选取首元素或随机位置也可以

快速排序

一趟排序

- 选出枢轴
- 两边往中间收缩填坑
- 分出大小区



快速排序

• 一趟排序算法

– 选出枢轴

```
int pivot = array[left];    // 枢轴值, 变成第一个坑  
int low = left, high = right;
```

– 往中间收缩直到分出大小区 `while (low < high) {`

– 右线收缩找到第一个错 `while (low < high && array[high] >= pivot)
 high--;`

– 填坑 `array[low] = array[high];`

– 左线收缩找到第一个错 `while (low < high && array[low] <= pivot)
 low++;`

– 填坑 `array[high] = array[low];`

– 区分完毕，枢轴回填 `}`

```
array[low] = pivot;
```

– 返回枢轴的下标 `return low;`

快速排序

- 算法

```
int pivotIndex = partition(array, left, right);
```

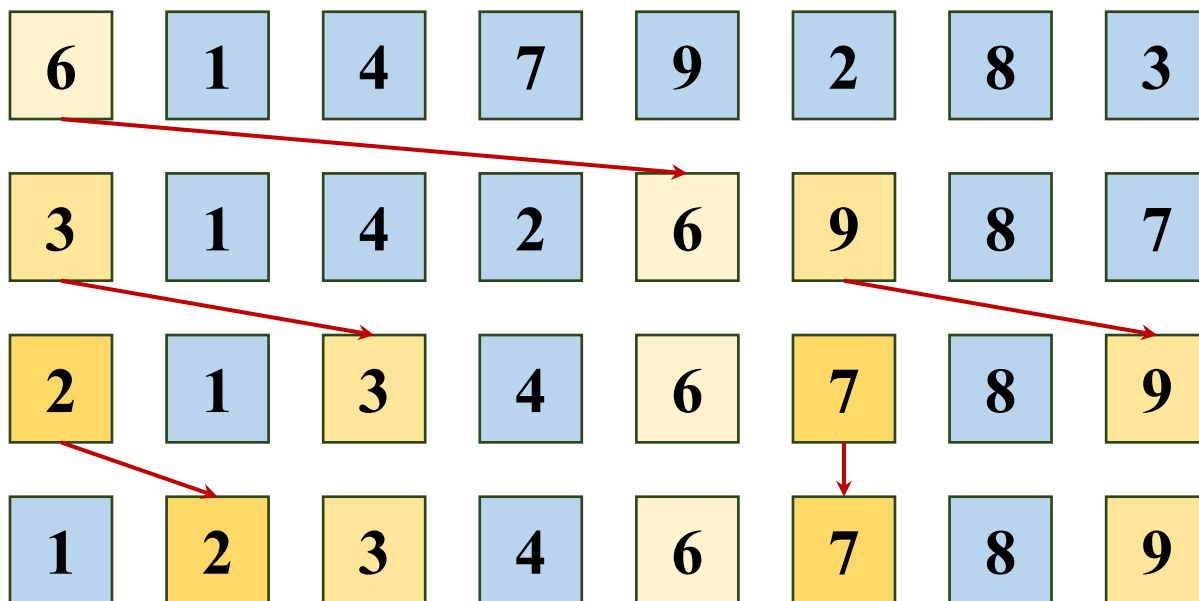
- 选出枢轴，将数组分为左右区

- 左区排序

```
quickSortRange(array, left, pivotIndex - 1);
```

- 右区排序

```
quickSortRange(array, pivotIndex + 1, right);
```



```
void quickSortRange(int array[], int left, int right) {
    if (left < right) {
        int pivotIndex = partition(array, left, right);
        quickSortRange(array, left, pivotIndex - 1);
        quickSortRange(array, pivotIndex + 1, right);
    }
}

void quickSort(int array[], int count) {
    if (count > 1) {
        quickSortRange(array, 0, count - 1);
    }
}
```

快速排序

- 假设一次划分所得枢轴位置 $i=k$ ，则对 n 个记录进行快速排序所需时间为

$$T(n) = T_{\text{pass}}(n) + T(k-1) + T(n-k)$$

- 若待排序列中的关键字随机分布，则 k 取 1 至 n 中任意一个值是等概率的。

快速排序

- 由此可得快速排序所需时间的平均值为：

$$T(n) = cn + \frac{1}{n} \sum_{k=1}^n (T(k-1) + T(n-k))$$

$$T(n) = cn + \frac{2}{n} \sum_{k=0}^{n-1} T(k) \quad T(n-1) = c(n-1) + \frac{2}{n-1} \sum_{k=0}^{n-2} T(k)$$

$$T(n) = \frac{2n-1}{n} c + \frac{n+1}{n} T(n-1)$$

$$T(n) < \frac{n+1}{n} T(1) + 2(n+1) \left(\frac{1}{2} + \dots + \frac{1}{n+1} \right) c$$

$$T(n) \leq \frac{n+1}{n} b + 2c(n+1) \ln(n+1) \leq 2d(n+1) \ln(n+1)$$

快速排序

- 若待排记录的初始状态按关键字有序，快速排序将蜕化为冒泡排序，其时间复杂度为 $O(n^2)$ 。
- 为避免出现这种情况，需在进行一次划分之前，进行“预处理”。例如，
- 先对 $L(s).key$ ， $L(t).key$ 和 $L[(s+t)/2].key$ 进行相互比较，然后取关键字为“三者之中”的记录作为枢轴记录。

目录

	3	分治法排序
	4	树形选择排序
	4.1	堆排序
	4.2	树形选择排序
	5	非比较排序

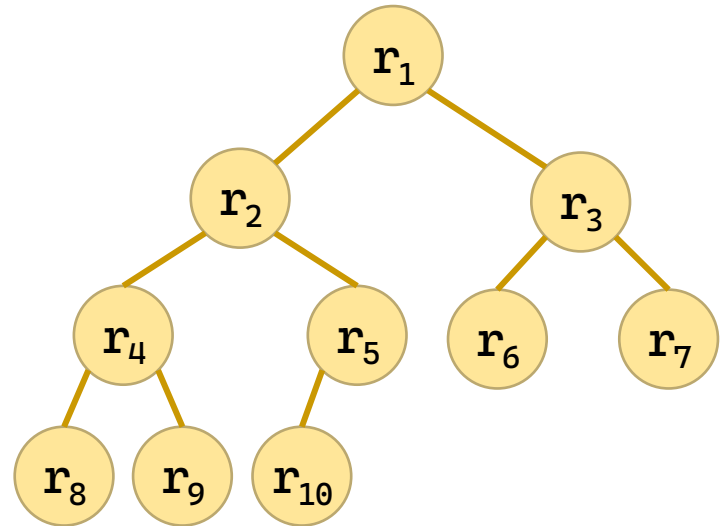
堆排序

- 堆是一棵完全二叉树。

- 结点 i 的孩子编号为 $2i$ 和 $2i+1$ ，父亲编号 $\lfloor i/2 \rfloor$
- 最大分支编号为 $n/2$ 。
- 树高为 $\lfloor \log_2 n \rfloor + 1$ 。

- 小顶堆和大顶堆

- 小顶堆 $r_i \leq r_{2i}, r_i \leq r_{2i+1}$
- 大顶堆 $r_i \geq r_{2i}, r_i \geq r_{2i+1}$

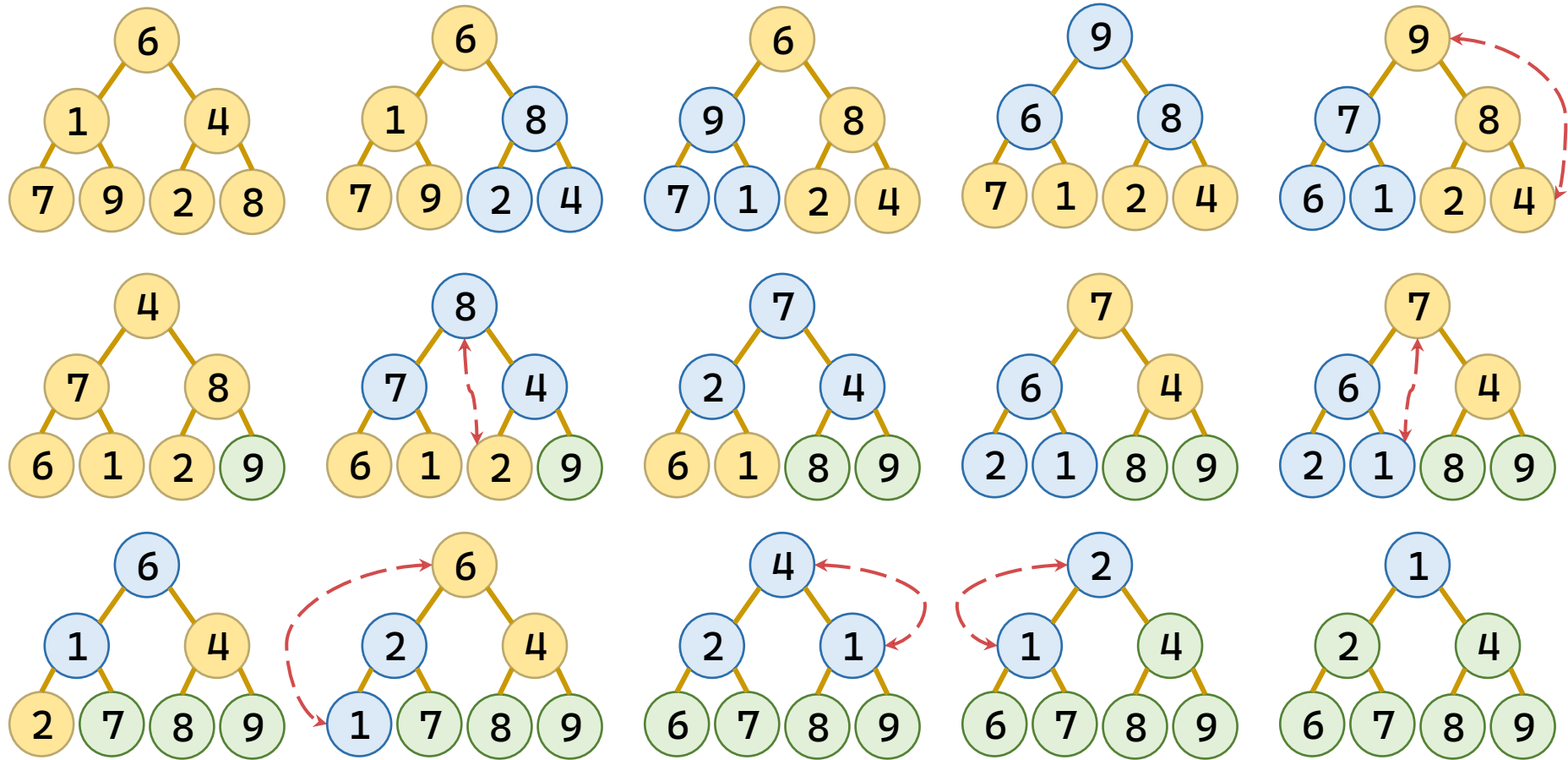


小顶堆: {12, 36, 27, 65, 40, 34, 98, 81, 73, 55}

不是堆: {12, 36, 27, 65, 40, 14, 98, 81, 73, 55}

堆排序

- 保持大顶堆（下滤操作）+提取最大值



堆排序

• 保持大顶堆（下滤操作）算法

– 自顶向下

– 找到三角的叶子最大值

▪ 如果有右子，比较左右

– 如果父子合规，停止

– 交换父子

– 向下过滤

```
while (root * 2 + 1 < count) {
```

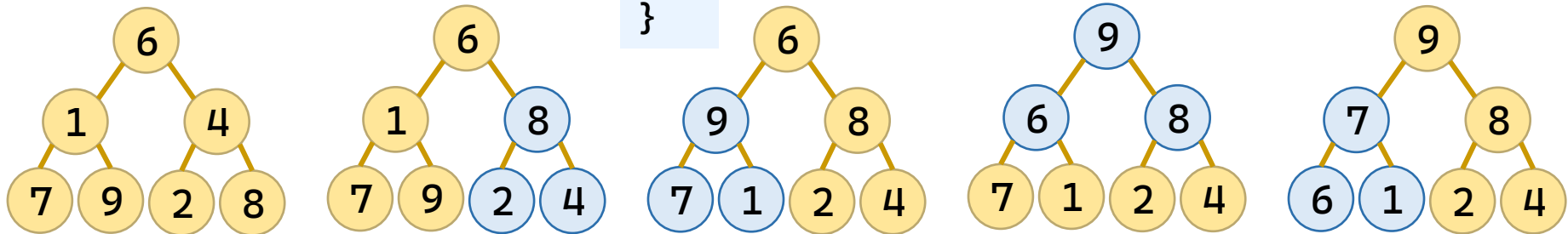
```
    int child = root * 2 + 1;  
    if (child + 1 < count  
        && array[child] < array[child + 1])  
        ++child;
```

```
    if (array[root] >= array[child])  
        return;
```

```
    swap(&array[root], &array[child]);
```

```
    root = child;
```

```
}
```



堆排序

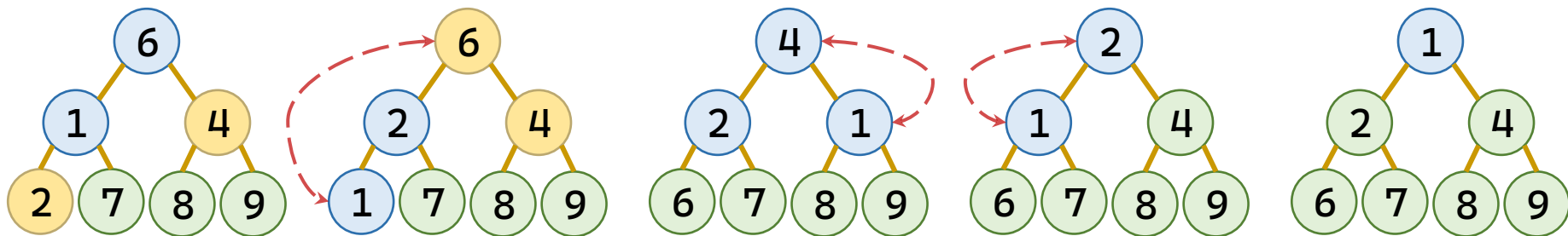
• 算法

- 第一趟：非叶子自底向上，依次下滤

```
for (index = count / 2 - 1; index >= 0; --index)
    siftDown(array, index, count);
```

- 第二趟：提取最顶（大）值，继续下滤

```
for (index = count - 1; index > 0; --index) {
    swap(&array[0], &array[index]);
    siftDown(array, 0, index);
}
```



```

void siftDown(int array[], int root, int count) {
    while (root * 2 + 1 < count) {
        int child = root * 2 + 1;
        if (child + 1 < count && array[child] < array[child+1])
            ++child;
        if (array[root] >= array[child])
            return;
        swap(&array[root], &array[child]);
        root = child;
    }
}

void heapSort(int array[], int count) {
    int index;
    for (index = count / 2 - 1; index >= 0; --index)
        siftDown(array, index, count);
    for (index = count - 1; index > 0; --index) {
        swap(&array[0], &array[index]);
        siftDown(array, 0, index);
    }
}

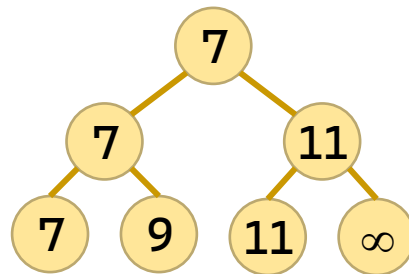
```

目录

	3	分治法排序
	4	树形选择排序
	4.1	堆排序
	4.2	树形选择排序
	5	非比较排序

树形选择排序

- 树形选择排序又称锦标赛排序，是一种按照锦标赛的思想进行选择排序的方法。首先对 n 个记录的关键字进行两两比较，然后在 $n/2$ 个较小者之间再进行两两比较，如此重复，直至选出最小的记录为止。
- 树形选择排序的思路来自于锦标赛/淘汰赛/选拔赛。
- 树形选择排序是一种在“完全二叉树”上完成的排序算法。
- 难点：如何生成和保存这棵完全二叉树？

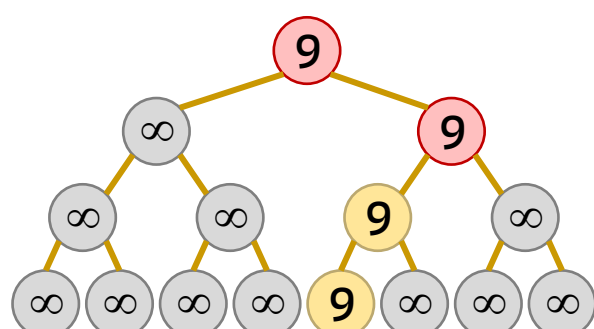
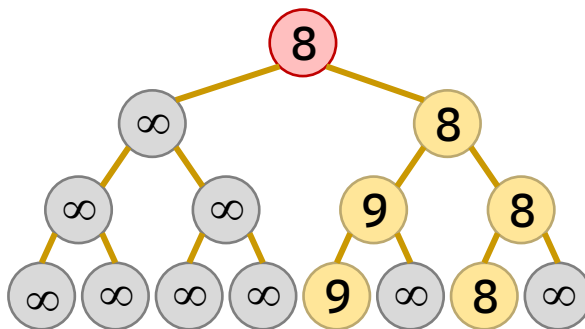
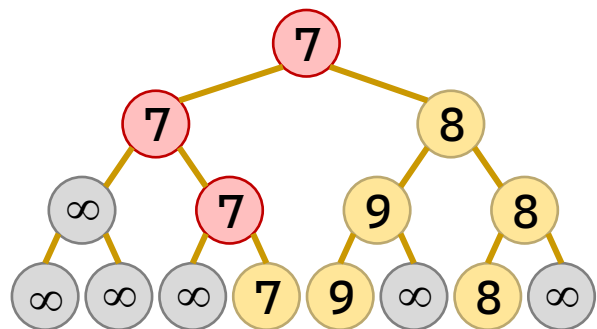
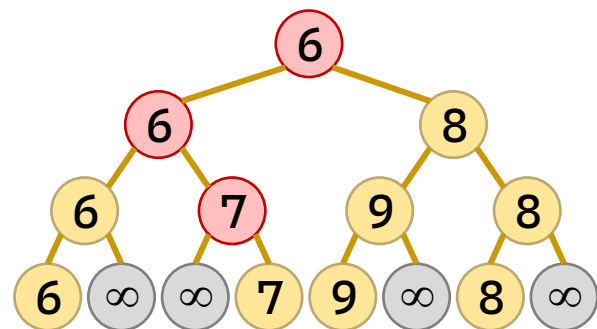
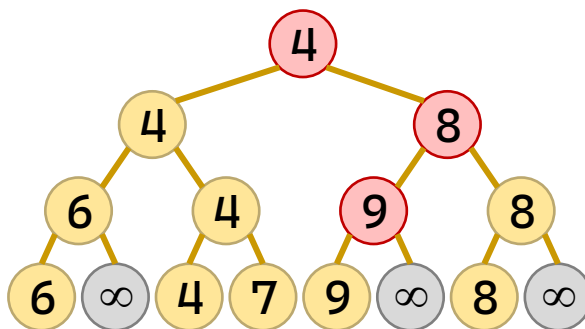
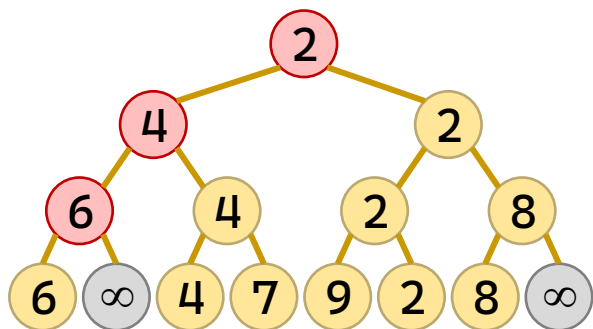
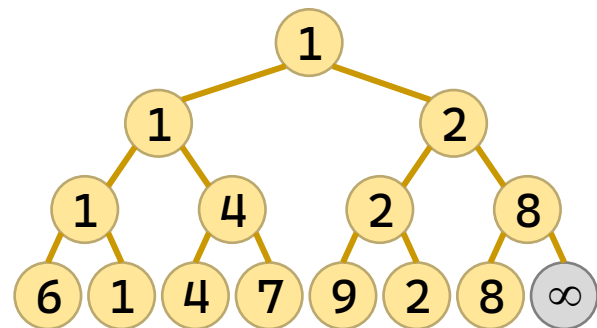


树形选择排序

• 算法

– 建树

– 提取根（最小值），维护树



树形选择排序

- 算法：建树

- 计算叶子数

```
int leafCount = 1;
while (leafCount < count)
    leafCount *= 2;
```

- 开辟空间

```
tree = (int*)malloc((size_t)(leafCount * 2) * sizeof(int));
candidates = (int*)malloc((size_t)leafCount * sizeof(int));
if (tree == NULL || candidates == NULL) {
    free(tree); free(candidates);
    perror("无法分配胜者树"); exit(EXIT_FAILURE);
}
```

- 叶子赋值

```
for (index = 0; index < leafCount; ++index) {
    candidates[index] = index < count ? array[index] : INT_MAX;
    tree[leafCount + index] = index;
}
```

- 父亲计算

```
for (index = leafCount - 1; index > 0; --index) {
    int left = tree[index * 2];
    int right = tree[index * 2 + 1];
    tree[index] = candidates[left] <= candidates[right] ? left : right;
}
```

树形选择排序

- 算法：提取根

- 提取根结点

```
for (iOut = 0; iOut < count; ++iOut) {  
    int winner = tree[1];  
    array[outputIndex] = candidates[winner];
```

- 从索引删去根结点

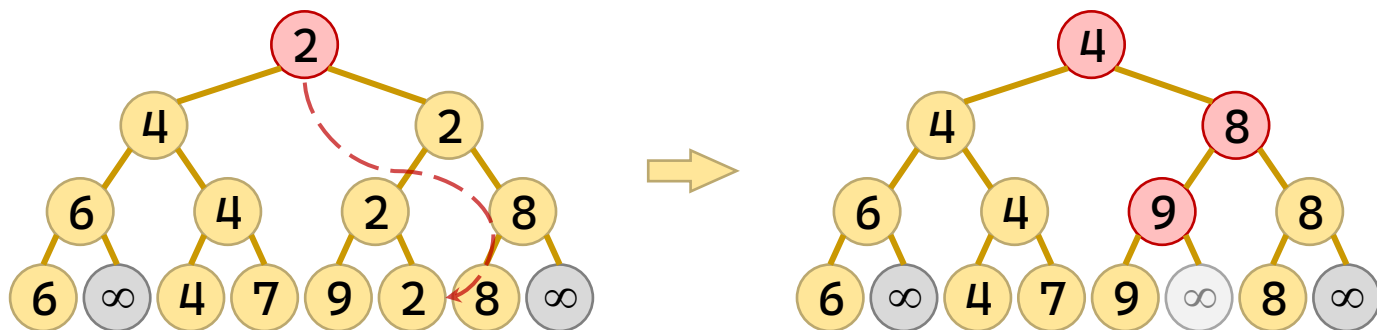
```
    candidates[winner] = INT_MAX;
```

- 计算根结点对应的叶子

```
    node = (leafCount + winner) / 2;
```

- 不断更新父亲

```
    while (node > 0) {  
        int l = tree[node*2], r = tree[node*2+1];  
        tree[node] = cand[l] <= cand[r] ? l : r;  
        node /= 2;  
    }
```



目录

	3	分治法排序
	4	树形选择排序
	5	非比较排序
	5.1	计数排序
	5.2	基数排序

计数排序

- 计数排序是一种计算式算法，要求数据元素为整型。
- 设数据元素(整数)的个数为 n ，数据元素最大值和最小值的差值为 $m-1$ 。
- 如果 $m < n \log_2 n$ ，计数排序算法的时间复杂度小于比较式排序算法。

计数排序

• 算法

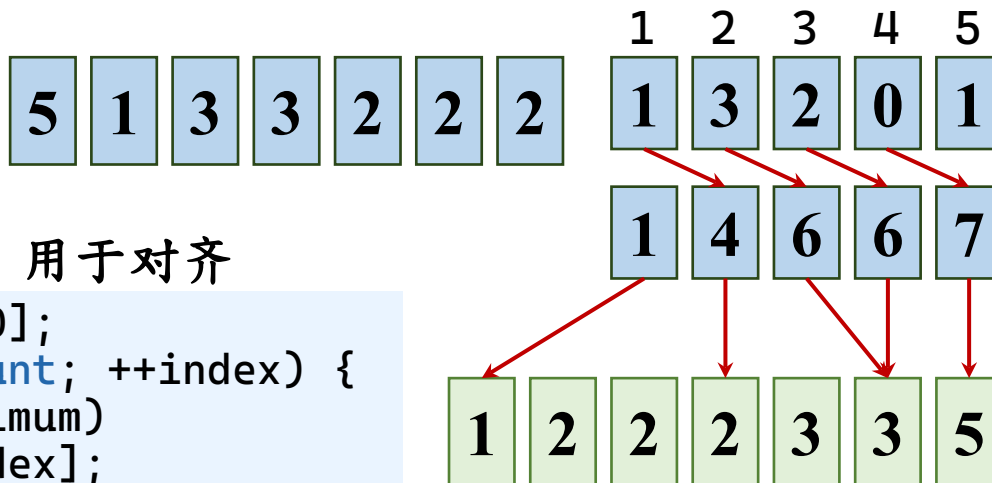
– 预备工作 (对齐)

- 算出最大最小值范围，用于对齐

```
minimum = maximum = array[0];
for (index = 1; index < count; ++index) {
    if (array[index] < minimum)
        minimum = array[index];
    if (array[index] > maximum)
        maximum = array[index];
}
range = maximum - minimum + 1;
```

- 开辟空间

```
counts = (int*)calloc((size_t)range, sizeof(int));
output = (int*)malloc((size_t)count * sizeof(int));
if (counts == NULL || output == NULL) {
    free(counts); free(output);
    perror("无法分配计数排序辅助数组");
    exit(EXIT_FAILURE);
}
```



计数排序

• 算法

– 计算统计直方图

```
for (index = 0; index < count; ++index)
    ++counts[array[index] - minimum];
```

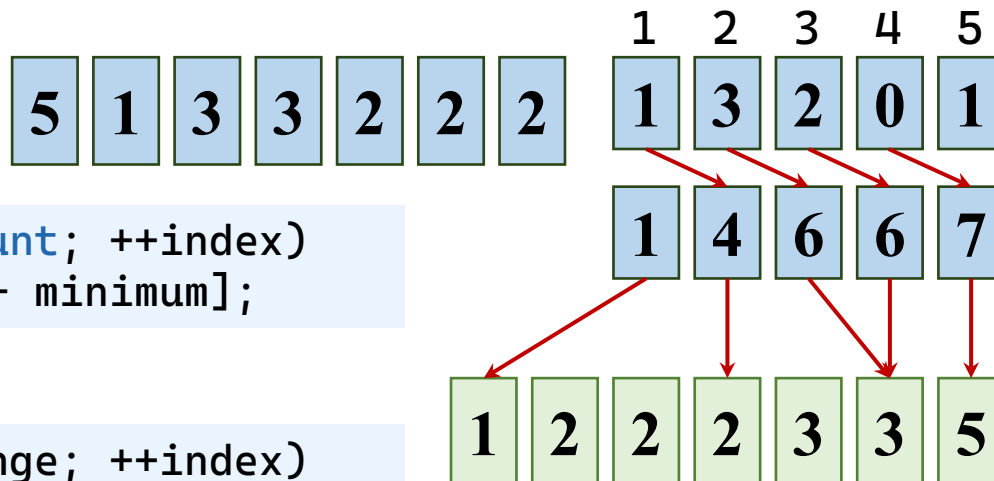
– 计算累计计数直方图

```
for (index = 1; index < range; ++index)
    counts[index] += counts[index - 1];
```

– 填入输出位置

- 数组：counts[]，偏移量：array[index] - minimum
- 填一个就回退一格

```
for (index = count - 1; index >= 0; --index)
    output[--counts[array[index] - minimum]] = array[index];
memcpy(array, output, (size_t)count * sizeof(int));
```



```

void countingSort(int array[], int count) {
    int minimum, maximum, range, index;
    int* counts, * output;
    if (count < 2)
        return;
    minimum = maximum = array[0];
    for (index = 1; index < count; ++index) {
        if (array[index] < minimum)
            minimum = array[index];
        if (array[index] > maximum)
            maximum = array[index];
    }
    range = maximum - minimum + 1;
    counts = (int*)calloc((size_t)range, sizeof(int));
    output = (int*)malloc((size_t)count * sizeof(int));
    if (counts == NULL || output == NULL) {
        free(counts);
        free(output);
        perror("无法分配计数排序辅助数组");
        exit(EXIT_FAILURE);
    }
}

```

```
for (index = 0; index < count; ++index)
    ++counts[array[index] - minimum];
for (index = 1; index < range; ++index)
    counts[index] += counts[index - 1];
for (index = count - 1; index >= 0; --index)
    output[--counts[array[index] - minimum]] = array[index];
memcpy(array, output, (size_t)count * sizeof(int));
free(counts);
free(output);
}
```

目录

	3	分治法排序
	4	树形选择排序
	5	非比较排序
	5.1	计数排序
	5.2	基数排序

基数排序

- 假如多关键字的记录序列中，每个关键字的取值范围相同(如十进制数)，则按 LSD (Least Significant Digit first，最低位优先法) 法进行排序时，可以采用“分配-收集”法，该方法不需要进行关键字之间的比较。
- 基数排序方法的基本思路是，将单关键字看成是由多个数位(或多个字符)构成的多关键字，并采用“分配-收集”的方法进行排序。

链式基数排序

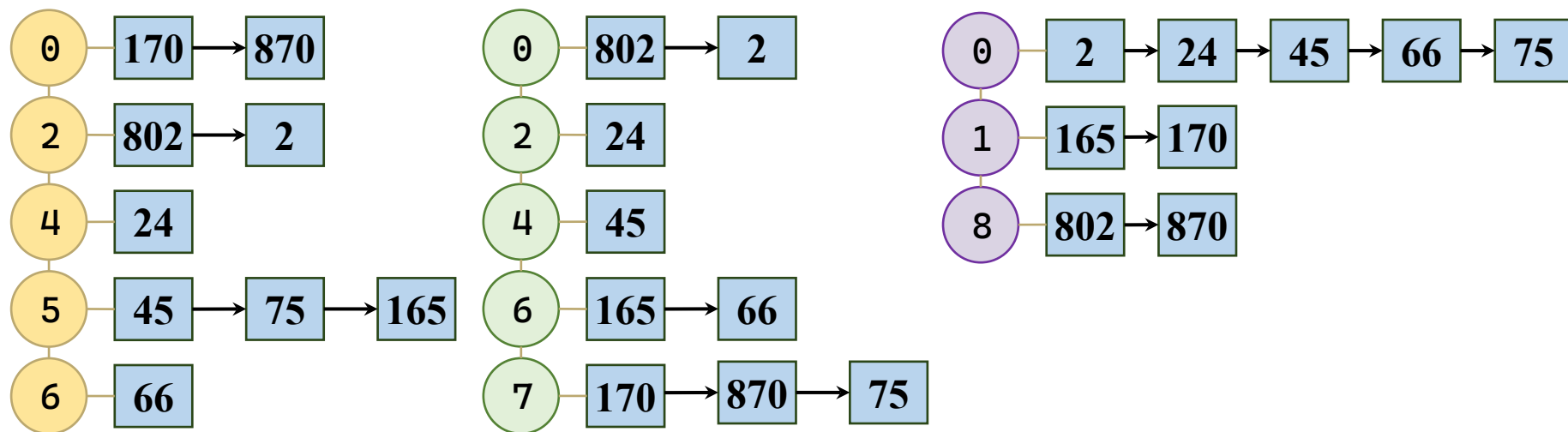
- 基数排序一般采用链表存储结构，即链式基数排序。
操作：
 - (1)将待排记录建成一个链表；
 - (2)分配时，将当前关键字位值相同的记录分配到同一个链队列中；
 - (3)收集时，按当前关键字位取值从小到大顺序将各链队列链成一个链表；
 - (4)对每个关键字位重复(2)和(3)两步。

链式基数排序

• 算法

170	45	75	870	802	24	2	66	165
-----	----	----	-----	-----	----	---	----	-----

- 按照数位遍历，把各数位的值相同的节点串联起来
- 第一遍，个位数小的到在前；第二遍，十位数小的排到前；第三遍，百位数小的排到前，十位数、个位数次序不变



链式基数排序

- 算法

- 按数位从小到大排序
- 初始化
- 遍历链表
- 找到对应的桶
进行尾插
- 将桶按顺序接续起来

```
int max = getMaxValue(*head);
for (exp = 1; max / exp > 0; exp *= 10) {

    RadixNode* bucketHead[10] = { NULL };
    RadixNode* bucketTail[10] = { NULL };
    RadixNode* current = *head;

    while (current != NULL) {
        int digit = (current->data / exp) % 10;
        RadixNode* nextNode = current->next;

        current->next = NULL;
        if (bucketTail[digit] == NULL) {
            bucketHead[digit] = current;
            bucketTail[digit] = current;
        } else {
            bucketTail[digit]->next = current;
            bucketTail[digit] = current;
        }

        current = nextNode;
    }
}
```

链式基数排序

- 算法

- 按数位从小到大排序
- 初始化
- 遍历链表
- 找到对应的桶
进行尾插
- 将桶按顺序接续起来
- 更新头指针

```
int max = getMaxValue(*head);  
for (exp = 1; max / exp > 0; exp *= 10) {
```

```
    for (int i = 0; i < 10; ++i)  
        if (bucketHead[i] != NULL)  
            if (newHead == NULL) {  
                newHead = bucketHead[i];  
                newTail = bucketTail[i];  
            } else {  
                newTail->next = bucketHead[i];  
                newTail = bucketTail[i];  
            }  
}
```

```
*head = newHead;
```

```

void radixSort(RadixNode** head) {
    if (head == NULL || *head == NULL)
        return;
    int max = getMaxValue(*head), exp;
    for (exp = 1; max / exp > 0; exp *= 10) { // 从个位到最高位
        /* 定义 10 个桶的头尾指针 */
        RadixNode* bucketHead[10] = {}, bucketTail[10] = {};
        RadixNode* current = *head;
        /* 分配阶段：遍历原链表，根据当前位数字放入对应的桶 */
        while (current != NULL) {
            int digit = (current->data / exp) % 10;
            RadixNode* nextNode = current->next; /* 下一个节点 */
            current->next = NULL; /* 断开当前节点 */
            /* 若桶为空，则头尾均指向该节点；否则追加到尾部 */
            if (bucketTail[digit] == NULL) {
                bucketHead[digit] = current;
                bucketTail[digit] = current;
            } else {
                bucketTail[digit]->next = current;
                bucketTail[digit] = current;
            }
            current = nextNode;
        }
    }
}

```

```

/* 收集阶段：按 0~9 的顺序将所有桶重新链接为一个链表 */
RadixNode* newHead = NULL;
RadixNode* newTail = NULL;
for (int i = 0; i < 10; ++i) {
    if (bucketHead[i] != NULL) {
        if (newHead == NULL) {
            newHead = bucketHead[i];
            newTail = bucketTail[i];
        } else {
            newTail->next = bucketHead[i];
            newTail = bucketTail[i];
        }
    }
}
*head = newHead; /* 更新头指针，进入下一次位处理 */
}
}

```

链式基数排序

- 基数排序算法的基本思路：将单关键字看成是由多个数位 (或多个字符)构成的多关键字，并采用 多趟 “分配-收集” 的方法进行排序。
- 算法时间复杂度为 $O(k(n+r))$ 。

9

谢谢观看

理论课程



廈門大學
XIAMEN UNIVERSITY



信息学院 黃 煒
(特色化示范性软件学院) 博士, 副教授
School of Informatics Wei Huang